

Sliding Window Maximum

September 10, 2023

📅 2023 · # rust, # algorithms · 📖 notes

The *Sliding Window Maximum* problem is a classic algorithmic challenge that involves processing an array in a sliding window fashion. The goal is to find the maximum element within a fixed-size window as it moves through the array.

Formally, the problem can be defined as follows.

Given an array $A[0, n - 1]$ and an integer k , the goal is to find the maximum of each subarray (window) of A of size k .

The simplest approach to address this problem involves handling each of the $n - k + 1$ windows independently. Within each window, we calculate its maximum by scanning through all its elements, which takes $\Theta(k)$ time. Consequently, this straightforward brute force solution operates in $\Theta(nk)$ time.

Here is a Rust implementation.

```
fn brute_force(v: &Vec<i32>, k: usize) -> Vec<i32> {
    let n = v.len();

    if k > n {
        return Vec::<i32>::new();
    }

    let mut maxs = Vec::with_capacity(n - k + 1);
    for i in 0..(n - k + 1) {
        let current_slice = &v[i..i + k];
        let max_value = *current_slice.iter().max().unwrap();
        maxs.push(max_value);
    }

    maxs
}
```

A more elegant one-line implementation uses [combinators](#).

```
fn brute_force(v: &Vec<i32>, k: usize) -> Vec<i32> {
    v.windows(k).map(|w| *w.iter().max().unwrap()).collect()
}
```

The inefficiency of this solution stems from the fact that when calculating the maximum of a window, we disregard all the computations previously performed to determine the maxima of the preceding windows.

BST-based Solution

Enhancing the brute force solution above entails leveraging a data structure to efficiently handle the next window while capitalizing on the progress made in processing the preceding window. The design of a faster solution begins with two straightforward observations:

Firstly, we can represent the elements within a window as a multiset $\mathcal{M}$ of size k . In this representation, the result for the window is essentially the largest element contained within this multiset.

Secondly, when we transition from one window to the next, only two elements change: the first element of the first window exits from the scene, and the last element of the second one enters. Consequently, we can derive the multiset of the new window from the multiset of the previous window by simply adding one element and removing another one.

Hence, we require a data structure capable of performing three crucial operations on a (multi)set: inserting a new element, deleting an arbitrary element, and efficiently retrieving the maximum element within the multiset. By employing such a versatile data structure, we can seamlessly move the window across the array while efficiently updating and querying the multiset to calculate the desired results. Now

the question is: What's the best data structure supporting these operations? A *Balanced Binary Search Tree* (BBST) supports any of these operations in $\Theta(\log |\mathcal{M}|)$, where $|\mathcal{M}|$ is the number of elements in the multiset (and it is optimal in the comparison model). This way, we can solve the problem in $\Theta(n \log k)$ time.

A Rust implementation of this strategy is as follows. Here we use [BTreeSet](#). A **BTreeSet** is BST-like data structure that represents a set of unique, ordered elements. It provides efficient implementation of insertion ([insert](#)), deletion ([remove](#)), and maximum ([last](#)) operations. The only issue to deal with is that a **BTreeSet** does not store repeated values. For this reason, we store elements together with their positions. This way, every element in the array is inserted as a unique pair in the **BTreeSet**.

```
use std::collections::BTreeSet;

pub fn bst(nums: &Vec<i32>, k: usize) -> Vec<i32> {
    let n = nums.len();
    if k > n {
        return Vec::<i32>::new();
    }

    let mut maxs = Vec::with_capacity(n - k + 1);

    let mut set = BTreeSet::new();
    let mut max_sf = nums[0];

    for (i, &v) in nums.iter().enumerate() {
        set.insert((v, i));

        // keep track of the max so far to avoid a costly query to the set
        max_sf = max_sf.max(v);

        if i >= k {
            set.remove(&(nums[i - k], i - k));
            if max_sf == nums[i - k] {
                max_sf = set.last().unwrap().0;
            }
        }
        if i >= k - 1 {
            maxs.push(max_sf);
        }
    }

    maxs
}
```

In this implementation we've incorporated a straightforward optimization to reduce the number of calls to `set.last()`. We keep track of the maximum element encountered so far, denoted as `max_sf`. We only call `set.last()` when `max_sf` might be invalidated by a deletion of an element equals to `max_sf`.

Heap-based Solution

It's worth noting an alternative solution that, theoretically, is slightly less efficient than the previous one (i.e., $\Theta(n \log n)$ instead of $\Theta(n \log k)$). However, in practice, this alternative solution often proves to be faster.

As we are talking about maximum, the immediate choice that springs to mind is the priority queue, with its most renowned manifestation being the *(max-)heap*. A max-heap stores a set n of keys and supports three operations:

- Insert an element in the max-heap. The operation takes $O(\log n)$ time. In Rust's implementation this is called [push](#).
- Report the maximum element in the max-heap. The operation takes $O(1)$ time. In Rust's implementation this is called [peek](#).
- Extract the maximum element from the max-heap. The operation takes $O(\log n)$ time. In Rust's implementation this is called [pop](#).

We can solve the sliding window maximum problem by employing a max-heap and scanning the array from left to right. Here's how it works.

Initially, we populate a max-heap with the first k elements of A along with their respective positions. This gives us the maximum within the initial window, which is essentially the maximum provided by the heap.

As we move on to process the remaining elements of A one by one, we insert each current element into the heap alongside its position. We then request the heap to provide us with the current maximum. However, it's important to note that this reported maximum element

maximum is within the constraints of the current window.

A Rust implementation of this strategy is the following one.

```
use std::collections::BinaryHeap;

pub fn heap(nums: &Vec<i32>, k: usize) -> Vec<i32> {
    let n = nums.len();

    if k > n {
        return Vec::<i32>::new();
    }

    let mut heap: BinaryHeap<i32, usize> = BinaryHeap::new();

    for i in 0..k - 1 {
        heap.push((nums[i], i));
    }

    let mut maxs = Vec::with_capacity(n - k + 1);

    for i in k - 1..n {
        heap.push((nums[i], i));
        while let Some((_, idx)) = heap.peek() {
            if *idx < i - (k - 1) {
                heap.pop();
            } else {
                break;
            }
        }
        maxs.push(heap.peek().unwrap().0);
    }
    maxs
}
```

It's worth noting that with this approach, there are a total of n insertions and at most n extractions of the maximum in the heap. Since the maximum number of elements present in the heap at any given time is up to n , each of these operations takes $\Theta(\log n)$ time. Consequently, the overall time complexity is $\Theta(n \log n)$.

Linear Time Solution

Can we achieve a better solution than the one presented earlier? Given that this section is titled *linear time solution*, you might rightly speculate, “Yes, it’s possible.” But, why is it intuitively reasonable to think about an improvement? Well, a good point is to observe that the BST-based solution can do much more than what is needed. If I ask you: *What’s the second largest element in the window?* No problem, the second largest element is the predecessor of the maximum and a BST supports also this operation in $\Theta(\log n)$ time. You would be able to report the top- x largest or smallest elements in $\Theta(x + \log n)$ time (How?). This is because the BST is implicitly keeping all the elements of all the windows sorted. The fact that we can do much more than what is requested, it’s an important signal to think that a faster solution could exist. Still, the title of this section is a stronger one.

Surprisingly, the better solution uses an elementary data structure: a queue. We require a *Double-Ended Queue* (Deque), which supports constant time insertion, removal and access at the front and the back of the queue. There are several ways to implement a deque. The easiest (but not the fastest) way is probably with a bidirectional list.

The algorithm starts with an empty deque Q and with the window W that covers the positions in the range $\langle -k, -1 \rangle$. That is, the window starts before the beginning of the array A . Then, we start sliding the window one position at a time and remove/insert elements from Q . We claim that the front of Q will be the element to report.

More precisely, we repeat n times the following steps.

- Move the window one position to the right
- Remove from the head of Q the elements that are no longer in the window
- Insert the new element from the tail of Q and remove all the elements above it until we find a larger element
- Report the head of Q as the maximum in the current window

The implementation of the above solution is the following.

```

use std::collections::VecDeque;

fn linear(nums: &Vec<i32>, k: usize) -> Vec<i32> {
    let n = nums.len();

    if k > n {
        return Vec::<i32>::new();
    }

    let mut q: VecDeque<usize> = VecDeque::new();
    let mut maxs: Vec<i32> = Vec::with_capacity(n - k + 1);

    for i in 0..k {
        while (!q.is_empty()) && nums[i] > nums[*q.back().unwrap()] {
            q.pop_back();
        }
        q.push_back(i);
    }
    maxs.push(nums[*q.front().unwrap()]);

    for i in k..n {
        while !q.is_empty() && q.front().unwrap() + k <= i {
            // more idiomatic while let Some(&(p,_)) = q.front()
            q.pop_front();
        }

        while (!q.is_empty()) && nums[i] > nums[*q.back().unwrap()] {
            q.pop_back();
        }
        q.push_back(i);
        maxs.push(nums[*q.front().unwrap()]);
    }

    maxs
}

```

Correctness

Let's prove the correctness of this solution. Looking at the running example below we enthusiastically think: "it could work"[cit](#). Why?

		0	1	2	3	4	5	6	7							
A		1	2	3	2	1	4	5	3							
	A[i]	i														
	1	0	2	1	3	2	3	2	3	2	4	5	5	6	5	6
Q							2	3	2	3					3	7
									1	4						
output							3	3	3	4	5	5				

A running example of the linear time solution with $k=3$. The deque stores a pair of values: a position in the array and the value at that position.

We first observe that *the elements in Q are always sorted in decreasing order*. This can be proved by induction on the number of iterations. The claim is true for the initial Q as it is empty. Given the queue after i iterations, by hypothesis, it is sorted. The current iteration will only remove elements (no change in the ordering of the remaining elements) or insert the current element $A[i + 1]$ as the tail of the queue just below the first element which is larger than it (if any). Thus, the queue remains sorted.

The sortedness of Q is a nice starting point for proving the correctness but it's not enough. We need now to introduce the definition of *right leaders* of the window to show that the largest element within the current window is at the top of the queue. Given a window, an element is called a right leader if and only if the element is larger than any other element of the window at its right.

As an example, consider the window of size 5 below.

0	1	2	3	4
4	2	3	1	1

The right leaders of this window are drawn in red.

We are now ready to prove a nice property of the elements in Q : *At every iteration, Q contains all and only the right leaders of the current window.*

This is quite easy to see. Firstly, any right leader cannot be removed from Q as all the subsequent elements are smaller than it. Secondly, any non-right leader will be removed as soon as the next right leader enters Q . Finally, any element outside the window cannot be in Q . By contradiction, let us assume that Q contains one such element, say a . Let r be the largest right leader. On the one hand, a cannot be smaller than or equal to r , otherwise a would be removed when inserting r in Q . On the other hand, a cannot be larger than r , otherwise, it would be in front of Q and removed by the first inner loop.

We derive the correctness of the algorithm by combining the sortedness of Q with the fact that the largest right leader is the element to report.

Time Complexity

Let us show that the algorithm runs in linear time.

We first use the standard approach to analyze an algorithm. We have a loop that is repeated n times. What's the cost of an iteration? Looking at the implementation it should be clear that its cost is dominated by the cost (and, thus, number) of pop operations. However, in a certain iteration, we may pop out all the elements in the deque. As far as we know there may be up to n elements in the deque and, thus, an iteration costs $O(n)$ time. So, the best we can conclude is that the algorithm runs in $O(n^2)$ time. Can't go too far with this kind of analysis!

In fact, there may indeed exist very costly iterations, but they are greatly amortized by many very cheap ones. Indeed, the overall number of pop operations cannot be larger than n as any element is not considered anymore by the algorithm as soon as it is removed from Q . Each of them costs constant time and, thus, the algorithm runs in linear time.

Next Larger Element

As an useful exercise, you could try to adapt the previous solution to solve the *Next Larger Element* problem, which is as follows.

Given an array $A[0, n - 1]$ having distinct elements, the goal is to find the next greater element for each element of the array in order of their appearance in the array.

These notes are for the "[Competitive Programming and Contests](#)" course at Università di Pisa.

Enjoy Reading This Article?

Here are some more articles you might like to read next:

- [HandsOn 3](#)
- [HandsOn 2](#)
- [HandsOn 1](#)
- [Mo's Algorithm](#)
- [Dynamic Prefix Sums with Fenwick Tree](#)

0 reactions



0 comments – powered by [giscus](#)

